

Cryptography Notes

With real world applications and examples

Eklavya Raman
er24ms024@iiserkol.ac.in

October 3, 2025

Contents

1	Basic Principles and Examples	3
1.1	Notation	3
1.2	Introduction	3
1.2.1	Historical Context	4
1.3	Attacks on Cryptographic Systems	5
1.3.1	Cryptographic Protocols	5
1.3.2	Provable Security	6
2	Symmetric Key Cryptography	9
2.1	Definitions and Overview	9
2.1.1	Stream Ciphers	10

List of Theorems

Chapter 1

Basic Principles and Examples

1.1 Notation

Definition 1.1.0.1

XOR (Exclusive OR) Operation

The XOR operation is denoted by the symbol $\oplus$. For two bits a and b , the XOR operation is defined as follows:

$$a \oplus b = \begin{cases} 1 & \text{if } a \neq b \\ 0 & \text{if } a = b \end{cases}$$

The truth table for the XOR operation is as follows:

a	b	$a \oplus b$
0	0	0
0	1	1
1	0	1
1	1	0

90

1.2 Introduction

Cryptography is the practice and study of techniques for secure communication in the presence of adversaries. It encompasses a wide range of methods and principles to ensure the confidentiality, integrity, authenticity, and non-repudiation of information. The fundamental goals of cryptography include:

- **Confidentiality:** Ensuring that information is accessible only to those authorized to have access.
- **Integrity:** Ensuring that information remains unaltered during storage or transmission.
- **Authentication:** Verifying the identity of users and devices.
- **Non-repudiation:** Preventing entities from denying their actions, such as sending a message.
- **Applications:** Cryptography is widely used in various applications, including secure communications (e.g., SSL/TLS), digital signatures, secure email, and blockchain technology.

Basic Principle of Cryptography: Cryptography relies on mathematical algorithms and protocols to transform plaintext (readable data) into ciphertext (unreadable data) and vice versa. The security of cryptographic systems often depends on the computational difficulty of certain mathematical problems, such as factoring large prime numbers or solving discrete logarithms.

Each cryptographic system typically involves an encryption algorithm, and a decryption algorithm. Each algorithm uses a key, with symmetric cryptography using the same key for both encryption and decryption, while asymmetric cryptography uses a pair of keys (public and private) for these processes. Let the plaintext be represented as P , the ciphertext as C , the encryption function as E , and the

decryption function as D . The relation between these elements for a symmetric key cryptographic system can be expressed as:

$$\begin{aligned} E(k, P) &= C \\ D(k, C) &= P \\ \implies D(k, E(k, P)) &= P \\ \implies E(k, D(k, C)) &= C \end{aligned}$$

Where k is the secret key used for both encryption and decryption. In asymmetric cryptography, the functions would be represented as:

$$\begin{aligned} E(k_{pub}, P) &= C \\ D(k_{priv}, C) &= P \\ \implies D(k_{priv}, E(k_{pub}, P)) &= P \\ \implies E(k_{pub}, D(k_{priv}, C)) &= C \end{aligned}$$

Where k_{pub} is the public key used for encryption and k_{priv} is the private key used for decryption.

1.2.1 Historical Context

Caesar Cipher - Symmetric Key Cryptography

The Caesar cipher is one of the earliest and simplest forms of encryption technique. It is a type of substitution cipher, where each letter in the plaintext is 'shifted' a certain number of places down or up the alphabet. For example, with a shift of 1, 'A' would be replaced by 'B', 'B' would become 'C', and so on. The key in this case is the number of positions each letter is shifted.

The Caesar cipher is named after Julius Caesar, who is reputed to have used this encryption method to protect his messages. Despite its historical significance, the Caesar cipher is not secure by modern standards, as it can be easily broken with frequency analysis or brute force attacks.

Let us define the algorithm mathematically. Let P be the plaintext letter, C be the ciphertext letter, and k be the shift key. The encryption and decryption functions can be defined as follows:

$$\begin{aligned} E(k, P) &= (P + k) \mod 26 \\ D(k, C) &= (C - k) \mod 26 \end{aligned}$$

Where P and C are represented as numbers from 0 to 25 (A=0, B=1, ..., Z=25).

Example 1.2.1.1 -

Let us take the plaintext "HELLO" and a shift key of 3, the encryption process would be:

$$\begin{aligned} H(7) &\rightarrow K(10) \\ E(4) &\rightarrow H(7) \\ L(11) &\rightarrow O(14) \\ L(11) &\rightarrow O(14) \\ O(14) &\rightarrow R(17) \end{aligned}$$

Thus, the ciphertext would be "KHOOR". To decrypt the message, we would apply the decryption function:

$$\begin{aligned} K(10) &\rightarrow H(7) \\ H(7) &\rightarrow E(4) \\ O(14) &\rightarrow L(11) \\ O(14) &\rightarrow L(11) \\ R(17) &\rightarrow O(14) \end{aligned}$$

Resulting in the original plaintext "HELLO".

1.3 Attacks on Cryptographic Systems

Cryptographic systems can be vulnerable to various types of attacks, which can compromise the confidentiality, integrity, and authenticity of the information they are designed to protect. Some common types of attacks include:

- **Brute Force Attack:** This involves systematically trying all possible keys until the correct one is found. The security of a cryptographic system against brute force attacks depends on the key length; longer keys provide greater security.
- **Frequency Analysis:** This attack exploits the statistical properties of the language used in the plaintext. By analyzing the frequency of letters or groups of letters in the ciphertext, an attacker can make educated guesses about the corresponding plaintext.
- **Chosen Plaintext Attack:** In this attack, the attacker can choose arbitrary plaintexts and obtain their corresponding ciphertexts. This can help the attacker to deduce the key or the encryption algorithm.
- **Ciphertext-Only Attack:** In this scenario, the attacker only has access to the ciphertext and must deduce the plaintext or the key without any knowledge of the corresponding plaintext. This type of attack is particularly challenging and requires a deep understanding of the encryption algorithm and the statistical properties of the plaintext.
- **Known Plaintext Attack:** In this attack, the attacker has access to both the plaintext and its corresponding ciphertext. This knowledge can be used to deduce the key or to find vulnerabilities in the encryption algorithm.
- **Chosen and Adaptive Chosen Ciphertext Attacks:** In these attacks, the attacker can choose ciphertexts and obtain their corresponding plaintexts. In an adaptive chosen ciphertext attack, the attacker can make multiple queries based on the results of previous queries, making it a more powerful form of attack.
- **Adaptive Chosen Plaintext Attack:** Similar to the chosen plaintext attack, but the attacker can adapt their choices based on previous results, making it more effective.

Kerckhoff's Principle

Kerckhoff's Principle is a fundamental concept in cryptography that states that a cryptographic system should be secure even if everything about the system, except the key, is public knowledge. This principle emphasizes the importance of key secrecy over the secrecy of the algorithm itself.

The principle was articulated by Auguste Kerckhoffs in the 19th century and has since become a cornerstone of modern cryptographic design. The main implications of Kerckhoff's Principle are:

- **Public Algorithms:** Cryptographic algorithms should be open to public scrutiny. This allows experts to analyze and test the algorithms for vulnerabilities, leading to more robust and secure systems.
- **Key Management:** The security of a cryptographic system relies heavily on the secrecy and management of keys. Proper key generation, distribution, storage, and destruction are critical to maintaining the overall security of the system.
- **Resilience to Attacks:** Since the algorithm is assumed to be known, the system must be designed to withstand various types of attacks, including brute force attacks, frequency analysis, and chosen plaintext attacks.

1.3.1 Cryptographic Protocols

We have the basic building blocks of cryptography, i.e., encryption and decryption algorithms, pseudo-random number generators, hash functions, etc. We now move on to the next level of abstraction, i.e., cryptographic protocols.

Definition 1.3.1.1**Cryptographic Protocol**

A cryptographic protocol is a sequence of operations that ensure secure communication and data exchange between parties. These protocols define how cryptographic algorithms and techniques are applied to achieve specific security goals, such as confidentiality, integrity, authentication, and non-repudiation.

Cryptographic protocols are essential for establishing secure channels over potentially insecure networks, such as the internet. They provide a framework for parties to communicate securely, even in the presence of adversaries who may attempt to intercept or manipulate the communication.

Example 1.3.1.1 -

Challenge and Response Protocol: This protocol is used for authentication purposes. When a user attempts to log in, the server sends a random challenge (a nonce) to the user. The user then encrypts this challenge using their private key and sends it back to the server. The server decrypts the response using the user's public key and verifies that it matches the original challenge. If it does, the user is authenticated. The protocol can be summarized as follows:

1. Server sends a random challenge C to the user.
2. User encrypts the challenge using their private key: $R = E(k_{priv}, C)$.
3. User sends the response R back to the server.
4. Server decrypts the response using the user's public key: $C' = D(k_{pub}, R)$.
5. Server verifies if $C' = C$. If true, the user is authenticated.

Since the set of possible challenges is large and randomly chosen, an attacker cannot easily guess the correct response without knowing the user's private key. This protocol ensures that only the legitimate user can respond correctly to the challenge, thereby providing a secure method of authentication.

1.3.2 Provable Security

Provable security is a rigorous approach to evaluating the security of cryptographic systems by providing mathematical proofs that demonstrate their resistance to various types of attacks. This approach aims to establish a formal relationship between the security of a cryptographic scheme and well-defined computational hardness assumptions.

A perfectly secure cryptographic system is one where the ciphertext provides no information about the plaintext without the key, even if the attacker has unlimited computational resources. However, perfect security is often impractical for real-world applications due to key management challenges and performance constraints. Therefore, most cryptographic systems aim for computational security, where breaking the system is computationally infeasible within a reasonable time frame.

Most provably secure cryptographic schemes are based on the assumption that certain mathematical problems are hard to solve. We define hardness of a problem as follows:

Definition 1.3.2.1**Hard Problem**

A problem is considered hard if there is no known polynomial-time algorithm that can solve it for all instances. In the context of cryptography, a problem is deemed hard if solving it would require an impractical amount of computational resources, making it infeasible for an attacker to break the cryptographic scheme within a reasonable time frame.

Common examples of hard problems used in cryptography include:

- **Integer Factorization Problem:** The difficulty of factoring large composite numbers into their prime factors is the basis for the security of RSA encryption.
- **Discrete Logarithm Problem:** The challenge of finding the exponent in the expression $g^x \bmod p$ given g and p is fundamental to the security of many public-key cryptosystems, such as Diffie-Hellman and ElGamal.

- **Elliptic Curve Discrete Logarithm Problem:** Similar to the discrete logarithm problem but defined over elliptic curves, this problem underpins the security of elliptic curve cryptography (ECC).

Provable security often involves reductions, where the security of a cryptographic scheme is shown to be at least as hard as solving a known hard problem. If an attacker could break the cryptographic scheme, they would also be able to solve the underlying hard problem, which is assumed to be infeasible.

Example 1.3.2.2 -

Consider the RSA encryption scheme, which is based on the integer factorization problem. The security of RSA relies on the assumption that factoring a large composite number n (the product of two large prime numbers p and q) is computationally infeasible. The RSA encryption and decryption processes can be summarized as follows:

- Key Generation:
 1. Choose two distinct large prime numbers p and q .
 2. Compute $n = p \times q$.
 3. Compute the totient $\phi(n) = (p - 1)(q - 1)$.
 4. Choose an integer e such that $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$.
 5. Compute d such that $d \equiv e^{-1} \pmod{\phi(n)}$.
 6. The public key is (e, n) and the private key is (d, n) .

- Encryption:

$$C = E(e, P) = P^e \pmod{n}$$

Where P is the plaintext message and C is the ciphertext.

- Decryption:

$$P = D(d, C) = C^d \pmod{n}$$

The security of RSA can be analyzed through a reduction to the integer factorization problem. If an attacker could efficiently decrypt messages without knowing the private key d , they would effectively be able to compute d from the public key (e, n) . This would require factoring n to find p and q , which is assumed to be a hard problem. Therefore, breaking RSA is at least as hard as factoring large composite numbers, providing a provable security guarantee based on the hardness of the integer factorization problem.

Definition 1.3.2.2

Random Oracle Model

The Random Oracle Model is a theoretical framework used in cryptography to analyze the security of cryptographic protocols and algorithms. In this model, a random oracle is an idealized black box that provides truly random responses to every unique query. The random oracle is assumed to be accessible to all parties, including potential adversaries, and it behaves like a perfect hash function that maps inputs to outputs in a completely random manner.

The Random Oracle Model is often used to prove the security of cryptographic schemes by demonstrating that if a scheme is secure in the random oracle model, it remains secure when the random oracle is instantiated with a real-world hash function. This approach allows cryptographers to analyze the security properties of protocols without relying on specific assumptions about the underlying hash functions, which may have vulnerabilities or weaknesses.

However, it is important to note that the Random Oracle Model is an idealization and does not perfectly represent real-world hash functions. While it provides valuable insights into the security of cryptographic schemes, there are known limitations and criticisms of the model, particularly regarding its applicability to practical implementations.

Definition 1.3.2.3**Cryptographic Hash Function**

A cryptographic hash function is a mathematical algorithm that takes an input (or 'message') and produces a fixed-size string of bytes, typically a digest that appears random. The output, known as the hash value or hash code, is unique to each unique input. Cryptographic hash functions are designed to be fast to compute, but infeasible to reverse (pre-image resistance), and even a small change in input should produce a significantly different output (avalanche effect). They are widely used in various applications, including data integrity verification, digital signatures, and password hashing. ***Ideal Hash Functions*** are defined to be truly random functions that map inputs to outputs in a way that is indistinguishable from a random oracle. In practice, real-world hash functions aim to approximate the properties of ideal hash functions as closely as possible.

We now move on to the study of symmetric key cryptography.

Chapter 2

Symmetric Key Cryptography

2.1 Definitions and Overview

A symmetric key cryptographic system is one where the same key is used for both encryption and decryption of messages. This means that both the sender and receiver must possess the same secret key, which must be kept confidential from any unauthorized parties. Symmetric key cryptography is also known as secret key cryptography or private key cryptography.

Symmetric key cryptography is widely used for secure communication due to its efficiency and speed compared to asymmetric key cryptography. However, it also presents challenges in key distribution and management, as the secret key must be securely shared between parties before communication can take place.

Definition 2.1.0.1

Symmetric Key Cryptographic Scheme

A symmetric key cryptographic scheme consists of a map -

$$E : K \times M \rightarrow C$$

such that for every key $k \in K$, the map $E(k, \cdot) : M \rightarrow C$ is a bijection. Here, K is the key space, M is the message space (or plaintext space), and C is the ciphertext space. The encryption function E takes a key and a message as input and produces a ciphertext as output. The decryption function $D : K \times C \rightarrow M$ is defined such that for every key $k \in K$, the map $D(k, \cdot) : C \rightarrow M$ is the inverse of the encryption function, i.e.,

$$D(k, E(k, m)) = m \quad \text{for all } m \in M$$

Note 2.1.0.1

The security of a symmetric key cryptographic scheme relies on the secrecy of the key k . If an attacker gains access to the key, they can easily decrypt any ciphertext produced using that key. Therefore, it is crucial to implement secure key management practices, including key generation, distribution, storage, and destruction.

Common examples of symmetric key cryptographic algorithms include the Advanced Encryption Standard (AES), Data Encryption Standard (DES), and Triple DES (3DES). These algorithms are widely used in various applications, including secure communications, data storage, and network security protocols.

Example 2.1.0.1 -

Let us construct a simple symmetric key cryptographic scheme using a substitution cipher. In this example, we will define the key space, message space, and ciphertext space, along with the encryption and decryption functions.

Key Space (K): The key space consists of all possible permutations of the alphabet. For simplicity,

we will use a small alphabet consisting of the letters A, B, C, D, and E. The key space K will include all possible arrangements of these letters. For example, one possible key could be the permutation $k = (E, D, C, B, A)$, which maps A to E, B to D, C to C, D to B, and E to A.

Message Space (M): The message space consists of all possible strings that can be formed using the letters A, B, C, D, and E. For example, a message could be "ABCD" or "EDE".

Ciphertext Space (C): The ciphertext space consists of all possible strings that can be formed using the letters A, B, C, D, and E. The ciphertext will be generated by applying the encryption function to the plaintext message using a specific key.

Encryption Function (E): The encryption function takes a key k and a message m as input and produces a ciphertext c as output. The encryption function can be defined as follows:

$$E(k, m) = c$$

where each letter in the message m is replaced by its corresponding letter in the key k . For example, if the key is $k = (E, D, C, B, A)$ and the message is "ABCD", the encryption process would be:

$$\begin{aligned} A &\rightarrow E \\ B &\rightarrow D \\ C &\rightarrow C \\ D &\rightarrow B \end{aligned}$$

Thus, the ciphertext would be "EDCB".

Decryption Function (D): The decryption function takes a key k and a ciphertext c as input and produces the original message m as output. The decryption function can be defined as follows:

$$D(k, c) = m$$

where each letter in the ciphertext c is replaced by its corresponding letter in the inverse of the key k . For the key $k = (E, D, C, B, A)$, the inverse key would be $k^{-1} = (A, B, C, D, E)$. Using the ciphertext "EDCB", the decryption process would be:

$$\begin{aligned} E &\rightarrow A \\ D &\rightarrow B \\ C &\rightarrow C \\ B &\rightarrow D \end{aligned}$$

Resulting in the original message "ABCD".

2.1.1 Stream Ciphers

Let K be the key space, M be the message space, and C be the ciphertext space. Let $k_1, k_2, \dots, k_n$ be a sequence of keys generated from the key space K . A stream cipher encrypts each symbol of the plaintext message $m = m_1 m_2 \dots m_n$ using a corresponding key symbol from the key sequence. The encryption function E can be defined as:

$$\begin{aligned} E(k_i, m_i) &= c_i \\ \implies c_i &= m_i \oplus k_i \end{aligned}$$

where c_i is the ciphertext symbol corresponding to the plaintext symbol m_i and the key symbol k_i . The overall ciphertext c is then formed by concatenating all the ciphertext symbols:

$$c = c_1 c_2 \dots c_n$$

The decryption function D is defined similarly, using the same key sequence to recover the original plaintext message:

$$\begin{aligned} D(k_i, c_i) &= m_i \\ \implies m_i &= c_i \oplus k_i^{-1} \end{aligned}$$

Note 2.1.1.1

Stream ciphers are particularly useful for encrypting data streams, such as audio or video transmissions, where data is processed in real-time. They are also commonly used in scenarios where the plaintext length is not known in advance, as they can handle variable-length messages efficiently.

A well-known example of a stream cipher is the RC4 algorithm, which was widely used in protocols such as SSL/TLS and WEP. However, due to various vulnerabilities discovered over time, RC4 is no longer considered secure for many applications. Modern stream ciphers, such as Salsa20 and ChaCha20, offer improved security and performance for contemporary cryptographic needs.

Example 2.1.1.2 -

Let us construct a simple stream cipher using the XOR operation for encryption and decryption. In this example, we will define the key space, message space, and ciphertext space, along with the encryption and decryption functions.

Key Space (K): The key space consists of all possible binary strings of a fixed length. For simplicity, we will use a key length of 4 bits. The key space K will include all possible 4-bit binary strings, such as "0000", "0001", "0010", ..., "1111".

Message Space (M): The message space consists of all possible binary strings of variable length. For example, a message could be "1101" or "101010".

Ciphertext Space (C): The ciphertext space consists of all possible binary strings of the same length as the message. The ciphertext will be generated by applying the encryption function to the plaintext message using a specific key.

Encryption Function (E): The encryption function takes a key k and a message m as input and produces a ciphertext c as output. The encryption function can be defined using the XOR operation as follows:

$$E(k, m) = c$$

where each bit in the message m is XORed with the corresponding bit in the key k . If the message is longer than the key, the key is repeated as necessary. For example, if the key is $k = "1010"$ and the message is "1101", the encryption process would be:

$$\begin{aligned} 1 \oplus 1 &= 0 \\ 1 \oplus 0 &= 1 \\ 0 \oplus 1 &= 1 \\ 1 \oplus 0 &= 1 \end{aligned}$$

Thus, the ciphertext would be "0111".

If the message is longer, for example "1101001", the key would be repeated as "1010101" and the encryption process would be:

$$\begin{aligned} 1 \oplus 1 &= 0 \\ 1 \oplus 0 &= 1 \\ 0 \oplus 1 &= 1 \\ 1 \oplus 0 &= 1 \\ 0 \oplus 1 &= 1 \\ 0 \oplus 0 &= 0 \\ 1 \oplus 1 &= 0 \end{aligned}$$

Resulting in the ciphertext "0111100".

Decryption Function (D): The decryption function takes a key k and a ciphertext c as input and produces the original message m as output. The decryption function can be defined using the XOR operation as follows:

$$D(k, c) = m$$

where each bit in the ciphertext c is XORed with the corresponding bit in the key k . Using the ciphertext "0111" and the key "1010", the decryption process would be:

$$0 \oplus 1 = 1$$

$$1 \oplus 0 = 1$$

$$1 \oplus 1 = 0$$

$$1 \oplus 0 = 1$$

Resulting in the original message "1101".

For the longer ciphertext "0111100" and the key "1010101", the decryption process would be:

$$0 \oplus 1 = 1$$

$$1 \oplus 0 = 1$$

$$1 \oplus 1 = 0$$

$$1 \oplus 0 = 1$$

$$1 \oplus 1 = 0$$

$$0 \oplus 0 = 0$$

$$0 \oplus 1 = 1$$

Resulting in the original message "1101001".

The above is an example of a simple stream cipher analogous to the one-time pad, which is theoretically unbreakable when used with a truly random key that is as long as the message and used only once. However, practical implementations often use pseudorandom key generators, which can introduce vulnerabilities if not properly designed.

Note 2.1.1.2

vulnerabilities of stream ciphers include:

- **Key Reuse:** If the same key is used for multiple messages, an attacker can exploit patterns in the ciphertexts to deduce information about the plaintexts. This is particularly problematic in stream ciphers, where the key stream must be unique for each encryption. How? Consider two plaintext messages P_1 and P_2 encrypted with the same key stream K :

$$C_1 = P_1 \oplus K$$

$$C_2 = P_2 \oplus K$$

An attacker who intercepts both ciphertexts can compute:

$$C_1 \oplus C_2 = (P_1 \oplus K) \oplus (P_2 \oplus K) = P_1 \oplus P_2$$

This reveals the XOR of the two plaintexts, which can provide significant information about the original messages, especially if one of the plaintexts is known or can be guessed.

- **Weak Key Generation:** If the key generation process is not truly random or is predictable, an attacker may be able to guess or derive the key, compromising the security of the encryption.
- **Statistical Attacks:** Stream ciphers can be vulnerable to statistical attacks if the key stream exhibits patterns or correlations that can be exploited by an attacker. This can occur if the pseudorandom number generator used to produce the key stream is not sufficiently secure.
- **Implementation Flaws:** Poor implementation practices, such as inadequate handling of keys or improper use of cryptographic libraries, can introduce vulnerabilities that attackers can exploit.

- **Transport of Keys:** Securely distributing and managing keys is a significant challenge in symmetric key cryptography. If keys are intercepted or leaked during transmission, the security of the entire system is compromised.